

Reporte de desarrollo y pruebas del proyecto JUEGUITO

Interfaz de entrada y salida de texto en C++
Versiones documentadas: 0.0.1 y 0.0.1.1

Equipo: Los Cholos
Grupo: 3A Mecatrónica

Programación Orientada a Objetos · 2026B
Mtro. Amed Aldama Bernabeu

Integrante	Código
Javier Mora Gutierrez	222432699
Erick Yael Ramos Ramirez	225975324
Luis Fernando Duarte Zàrate	225975219
Christian Osvaldo Guzman Orozco	221527157
Omar Contreras Paz	225975421
Derek Salcido González	225975499

1. Introducción

El proyecto JUEGUITO desarrolla una base para la interacción entre un jugador y un programa de consola. Su primera etapa se concentra en mostrar mensajes y recibir opciones numéricas, funciones necesarias para construir posteriormente menús, diálogos y decisiones dentro de un videojuego.

Este reporte integra los documentos de las versiones 0.0.1 y 0.0.1.1, el primer reporte del tester y el archivo jueguito.cpp. Describe la evolución del diseño, el funcionamiento de la implementación

y los hallazgos de su revisión. El archivo proporcionado corresponde a un prototipo de interfaz; todavía no implementa un videojuego completo.

2. Objetivos

Objetivo general. Analizar el desarrollo y la validación de una interfaz de usuario en C++ que permita mostrar texto y recibir únicamente opciones enteras dentro de un rango definido.

Objetivos específicos. Identificar la función de los métodos de entrada y salida; explicar su organización en una clase; documentar el error informado por el tester; y contrastar ese reporte con el comportamiento del código adjunto.

3. Evolución del proyecto

Versión 0.0.1, 3 de septiembre de 2026. La propuesta establece las funciones `Output()` e `Input()`. La primera muestra una cadena de texto; la segunda recibe un límite inferior, un límite superior y una pregunta. La entrada debe repetirse hasta obtener un número permitido.

Versión 0.0.1.1, 11 de septiembre de 2026. Se propone integrar ambas funciones como métodos de una clase llamada `InterfazUsuario`. Este cambio organiza las responsabilidades de presentación y lectura dentro de un objeto reutilizable.

Implementación entregada. El código emplea el nombre `UserInterface`, conserva la opción elegida en un atributo privado y agrega `GetOption()` para consultarla. A diferencia de la especificación, `Input()` está declarado como `void`: almacena el resultado, pero no lo devuelve directamente.

4. Aplicación de programación orientada a objetos

La clase encapsula el atributo `Option` y expone métodos públicos para interactuar con él. Su constructor inicializa ese atributo en cero. La instrucción `UserInterface UI;` crea un objeto de la clase; técnicamente se trata de instanciación, no de herencia. El prototipo evidencia encapsulamiento y uso de objetos, sin implementar herencia ni polimorfismo.

5. Funcionamiento del código proporcionado

Elemento	Responsabilidad
<code>Option</code>	Atributo privado de tipo <code>int</code> que almacena la opción.
<code>UserInterface()</code>	Constructor que inicializa <code>Option</code> en 0.
<code>Output(const std::string& Text)</code>	Imprime el texto y un salto de línea.
<code>Input(int, int, const std::string&)</code>	Muestra la pregunta y las opciones; lee y valida la entrada.
<code>GetOption()</code>	Devuelve el valor almacenado en <code>Option</code> .

En `main()` se crea el objeto UI y se imprime un mensaje de bienvenida. Después se solicita una opción del 1 al 3, se muestra el valor elegido y se presenta una segunda pregunta con opciones de -1 a 2.

`Input()` utiliza un ciclo `do-while` para repetir la solicitud mientras el valor esté fuera del rango. Cuando la lectura falla o quedan caracteres antes del salto de línea, ejecuta `cin.clear()` y `cin.ignore()` para recuperar el flujo y descartar la entrada restante.

El argumento `const std::string&` permite recibir texto por referencia constante, evitando una copia y su modificación dentro del método. El comentario original que describe `&` como un puntero es impreciso: en esta declaración representa una referencia.

6. Incidencia reportada por el tester: BUG-001

Descripción del reporte. El tester señala que una cadena de más de 100 caracteres, seguida de un espacio y un número válido, deja datos residuales en el búfer. Según su evidencia, estos datos provocan repeticiones automáticas y la aceptación de un número sin una nueva interacción.

Procedimiento descrito. Ejecutar el programa; escribir más de 100 letras en la primera pregunta; agregar un espacio y el número 2; y presionar Enter.

Resultado esperado. Descartar completamente la línea inválida, avisar al usuario y esperar una nueva opción. El número ubicado al final de esa misma línea no debe interpretarse como una respuesta independiente.

Impacto. Si ocurre, el fallo afecta el control del usuario sobre las decisiones y la confiabilidad de los menús. Por ello es necesario comprobar tanto la validación del número como el consumo completo de la entrada.

7. Contraste del reporte y revisión técnica

Se compiló el archivo adjunto con `g++` en modo `C++11` y advertencias activadas. Para contrastar BUG-001, se suministró una línea de 150 letras seguida de « 2», después una nueva línea con 3 y finalmente otra con 0. El programa descartó la primera línea y aceptó 3 como la primera selección.

Resultado: BUG-001 no se reprodujo en esa prueba sobre el archivo proporcionado. La implementación ya usa `std::numeric_limits<std::streamsize>::max()` al descartar caracteres, por lo que no presenta un límite fijo de 100. No se puede afirmar que la evidencia del tester corresponda exactamente a esta misma revisión del código o entorno.

Otros hallazgos. La inspección identifica que el fin de la entrada no tiene una salida específica del ciclo; al llegar a EOF, el programa puede repetir indefinidamente las preguntas. También rechaza espacios después de un entero válido y no explica por qué una respuesta fue rechazada. Además, conviene validar los límites y evitar operaciones que desborden un `int` en valores extremos.

8. Propuesta de mejora y comprobación local

Como parte de esta revisión se preparó una variante local que lee líneas completas mediante `getline()` y valida su contenido con `istringstream`. Acepta un único entero con espacios alrededor, rechaza contenido adicional, informa entradas inválidas y valores fuera de rango, y maneja el cierre de la entrada. También valida los límites y devuelve la opción desde `Input()`, conservando `GetOption()`.

La variante compiló con `-Wall -Wextra -Werror -pedantic`. Se ejecutaron 11 casos automatizados: entrada normal, texto de 10 000 caracteres, letras, decimales, varios valores, línea vacía, entero demasiado grande, opción fuera de rango, espacios alrededor del número, EOF y última línea sin salto final. Todos superaron las comprobaciones ejecutadas. Estas pruebas son de consola y no sustituyen una nueva validación interactiva del tester.

9. Conclusión

El prototipo cumple la función básica de mostrar texto y restringir opciones numéricas mediante una clase. La revisión resalta la importancia de mantener coherencia entre especificación, código y evidencia de pruebas. Aunque BUG-001 no se reprodujo en el archivo adjunto, se identificaron oportunidades concretas para mejorar el tratamiento de la entrada, los mensajes y la terminación del programa.

10. Fuentes del reporte

JueguitoV0-0-1.pdf. Especificación inicial, 03/09/2026.

JueguitoV0-0-1-1, Documentación.pdf. Diseño de la clase, 11/09/2026.

1er_reporte_tester.pdf. Reporte de Javier Mora Gutiérrez, BUG-001. `jueguito.cpp`.

Código adjunto y revisión técnica realizada para este reporte.